Région Hauts-de-France
Cofinancé par l'Union européenne
realconseil DIAGNOSTIC ACCOMPAGNEMENT FORMATION

Documentation Symfony -> Servir des ressources statiques via contrôleurs

Ce document présente une méthode simple et robuste pour intégrer des pages et des ressources "statiques" dans un projet Symfony, puis les servir via des contrôleurs.

On couvre 2 modes:

- **Partie A**: on sert via contrôleurs, sans rôles (tout est public)
- **Partie B**: on protège certains chemins avec authentification + rôles

---

1 Rappel: qui sert quoi, Apache/Nginx ou Symfony ?

1.1 Servir directement depuis `public/`

Quand on place un fichier dans `public/`, le serveur web (Apache/Nginx) peut le servir **sans passer par Symfony**.

- Avantages: rapide, simple, aucune exécution PHP.
- Exemple: `public/depot/files/doc.pdf` devient accessible directement via `/depot/files/doc.pdf`.

Ce mode convient parfaitement pour une vitrine 100% publique.

1.2 Servir via Symfony (contrôleurs)

Quand on sert un fichier via un contrôleur Symfony, on fait une URL "propre" et on ajoute du contrôle.

On peut alors:

- proposer des URLs propres (ex: `/p/hello` au lieu de `/depot/pages/hello.html`)
- vérifier ce qu'on autorise (extensions, chemins, etc.)
- gérer le contrôle d'accès (Partie B)
- avoir des logs uniformes (tout passe par l'application)
- bloquer les attaques classiques (path traversal avec `../`)

1.3 Point important de sécurité

Si un fichier est dans `public/`, il reste **accessible directement** par URL, même si on a créé un contrôleur qui le sert aussi.

Donc, pour du contenu "réservé", on choisit l'une de ces stratégies:

- **Stratégie 1 (recommandée)**: on stocke le contenu privé **hors de `public/`** (ex: `var/private/`, `/srv/private/`)
- **Stratégie 2 (possible)**: on stocke dans `public/` mais on **interdit l'accès direct** à ce dossier côté Apache, et on sert uniquement via Symfony

---

2 Organisation de répertoires recommandée

2.1 Dépôt stagiaires (public)

On peut autoriser le dépôt de contenus dans un dossier dédié de `public/`.

Exemple:

```
public/
  depot/
    pages/        (pages HTML)
    files/        (pdf, images)
    audio/        (mp3, ogg, wav)
```

Exemples de fichiers:

- `public/depot/pages/hello.html`
- `public/depot/files/reglement.pdf`
- `public/depot/audio/demo.mp3`

2.2 Espace privé (Partie B)

Option 1: privé dans le projet Symfony

Ici, `var/` est bien le **répertoire `var/` du projet Symfony** (au même niveau que `src/`, `public/`, etc.). Comme il n'est pas dans `public/`, son contenu n'est pas accessible directement via le web.

```
var/
  private/
    pages/
    files/
    audio/
```

Option 2: privé côté serveur (convention Debian)

Ici, `/srv/` est bien le **répertoire `/srv` à la racine du système** (convention Debian/Linux). On sépare alors les contenus privés du code applicatif.

```
/srv/private/
  pages/
```

```
    files/
    audio/
```

---

3 Les risques classiques et comment les éviter

3.1 Path traversal (attaque en `../`)

Quand on met un paramètre dans l'URL (ex: `/f/{filename}`) et qu'on l'utilise pour construire un chemin de fichier, on ouvre un risque: **on peut essayer de sortir du dossier autorisé**.

Exemple: si on construit le chemin naïvement:

- on reçoit `filename = ../../.env`
- on construit `public/depot/files/../../.env`
- le système résout le chemin, et on finit par lire un fichier sensible du projet

Donc on peut tenter:

- `/f/../../.env`
- `/f/../../../etc/passwd` (selon les cas)

Objectif: on garantit que le fichier final servi reste obligatoirement dans le dossier prévu (ex: `public/depot/files`).

Protections à cumuler:

- **1) On limite le paramètre avec une whitelist (regex)** On autorise uniquement les caractères attendus (lettres, chiffres, tirets, underscore, point). On refuse tout le reste (dont `/` et les séquences dangereuses).

- **2) On utilise `realpath()` et on vérifie l'emplacement final** On calcule:

  - `realBase = realpath(dossier_autorisé)`
  - `realFile = realpath(fichier_demandé)`

  Puis on vérifie que `realFile` commence par `realBase/`. Si ce n'est pas le cas, on refuse (404 ou 403).

En résumé: **même si quelqu'un essaye un `../`, on s'assure que le fichier final ne sort jamais du répertoire autorisé.**

3.2 Extensions non autorisées

Même avec un chemin correct, on ne veut pas servir n'importe quel type de fichier. On décide donc, pour chaque route, **quelles extensions on accepte**.

Exemples:

- pages: `.html` uniquement
- fichiers: `.pdf`, `.png`, `.jpg`, `.webp`, etc.

- audio: `.mp3`, `.ogg`, `.wav`

Objectif: éviter de servir par erreur:

- des fichiers de configuration
- des archives
- des scripts
- ou tout contenu qui n'a rien à faire dans une vitrine

Donc, dans le contrôleur, on récupère l'extension (`pathinfo`) et:

- si l'extension n'est pas dans la liste autorisée, on refuse.

exemple de controleur :

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\BinaryFileResponse;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

final class StaticFileController extends AbstractController
{
    #[Route('/f/{filename}', name: 'static_file', requirements: ['filename'
=> '[a-zA-Z0-9\-_\.]+' ])]
    public function file(string $filename): Response
    {
        // 1) Dossier autorisé
        $projectDir = $this->getParameter('kernel.project_dir');
        $baseDir = $projectDir . '/public/depot/files';

        // 2) Construit le chemin demandé
        $requestedPath = $baseDir . '/' . $filename;

        // 3) Vérifie que le fichier demandé reste bien dans le dossier
autorisé (anti ../)
        $realBase = realpath($baseDir);
        $realFile = realpath($requestedPath);

        if ($realBase === false || $realFile === false) {
            throw $this->createNotFoundException('Fichier introuvable.');
        }

        if (!str_starts_with($realFile, $realBase . DIRECTORY_SEPARATOR)) {
            // Tentative de sortie du dossier autorisé
            throw $this->createNotFoundException('Fichier introuvable.');
        }

        // 4) Vérifie l'existence et la lisibilité
        if (!is_file($realFile) || !is_readable($realFile)) {
```

```
            throw $this->createNotFoundException('Fichier introuvable.');
        }

        // 5) Whitelist des extensions autorisées
        $allowedExtensions = [
            'pdf', 'png', 'jpg', 'jpeg', 'webp', 'gif',
        ];

        $ext = strtolower(pathinfo($realFile, PATHINFO_EXTENSION));

        if (!in_array($ext, $allowedExtensions, true)) {
            // On refuse de servir une extension non prévue
            throw $this->createAccessDeniedException('Extension non
autorisée.');
        }

        // 6) Sert le fichier
        return new BinaryFileResponse($realFile);
    }
}
```

3.3 Ne jamais exécuter du code depuis un dépôt public

Un dépôt public (ex: `public/depot/`) doit rester un dépôt de contenu, pas un endroit où du code serveur peut s'exécuter.

Risque: si on autorise des fichiers `.php` dans un dossier servi par Apache, et que la configuration le permet, alors **un fichier PHP pourrait être exécuté** au lieu d'être servi comme un simple fichier.

Bonnes pratiques:

- **On interdit explicitement l'exécution de PHP** dans les dossiers de dépôt (côté Apache), par exemple en refusant les `.php`.
- **On refuse l'upload de `.php`** (si un jour on ajoute une fonctionnalité d'upload). Même si Apache est correctement configuré, on évite de laisser entrer ces fichiers.

En résumé: **dans un dépôt public, on n'autorise que du contenu (HTML, images, PDF, audio), jamais du code exécutable côté serveur.**

---

4 Mise en place Symfony: routes + contrôleurs

Les exemples ci-dessous utilisent les attributs Symfony (PHP 8+).

4.1 Route A: servir une page HTML depuis `public/depot/pages`

URL finale souhaitée:

- `/p/hello` -> sert `public/depot/pages/hello.html`

Contrôleur `StaticPageController.php`

```
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\BinaryFileResponse;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Attribute\Route;

final class StaticPageController extends AbstractController
{
    #[Route('/p/{slug}', name: 'static_page', requirements: ['slug' => '[a-
zA-Z0-9\-_]+' ])]
    public function page(string $slug): Response
    {
        $projectDir = $this->getParameter('kernel.project_dir');
        $baseDir = $projectDir . '/public/depot/pages';
        $file = $baseDir . '/' . $slug . '.html';

        $realBase = realpath($baseDir);
        $realFile = realpath($file);

        if ($realBase === false || $realFile === false) {
            throw $this->createNotFoundException('Page introuvable.');
        }

        if (!str_starts_with($realFile, $realBase . DIRECTORY_SEPARATOR)) {
            throw $this->createNotFoundException('Page introuvable.');
        }

        if (!is_file($realFile) || !is_readable($realFile)) {
            throw $this->createNotFoundException('Page introuvable.');
        }

        return new BinaryFileResponse($realFile);
    }
}
```

4.2 Route A: servir un fichier depuis `public/depot/files`

URL finale:

- `/f/reglement.pdf` -> sert `public/depot/files/reglement.pdf`

Contrôleur `StaticFileController.php`

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\BinaryFileResponse;
use Symfony\Component\Routing\Attribute\Route;

final class StaticFileController extends AbstractController
{
    #[Route('/f/{filename}', name: 'static_file', requirements: ['filename'
=> '[a-zA-Z0-9\-_\.]+' ])]
    public function file(string $filename): BinaryFileResponse
    {
        $projectDir = $this->getParameter('kernel.project_dir');
        $baseDir = $projectDir . '/public/depot/files';
        $file = $baseDir . '/' . $filename;

        $realBase = realpath($baseDir);
        $realFile = realpath($file);

        if ($realBase === false || $realFile === false) {
            throw $this->createNotFoundException('Fichier introuvable.');
        }

        if (!str_starts_with($realFile, $realBase . DIRECTORY_SEPARATOR)) {
            throw $this->createNotFoundException('Fichier introuvable.');
        }

        if (!is_file($realFile) || !is_readable($realFile)) {
            throw $this->createNotFoundException('Fichier introuvable.');
        }

        $allowed = ['pdf', 'png', 'jpg', 'jpeg', 'webp', 'gif'];
        $ext = strtolower(pathinfo($realFile, PATHINFO_EXTENSION));
        if (!in_array($ext, $allowed, true)) {
            throw $this->createAccessDeniedException('Extension non
autorisée.');
        }

        return new BinaryFileResponse($realFile);
    }
}
```

4.3 Route A: servir un audio (player)

URL:

- `/audio/demo.mp3` -> sert `public/depot/audio/demo.mp3`

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\BinaryFileResponse;
use Symfony\Component\Routing\Attribute\Route;

final class AudioController extends AbstractController
{
    #[Route('/audio/{filename}', name: 'audio_file', requirements:
['filename' => '[a-zA-Z0-9\-_\.]+' ])]
    public function audio(string $filename): BinaryFileResponse
    {
        $projectDir = $this->getParameter('kernel.project_dir');
        $baseDir = $projectDir . '/public/depot/audio';
        $file = $baseDir . '/' . $filename;

        $realBase = realpath($baseDir);
        $realFile = realpath($file);

        if ($realBase === false || $realFile === false) {
            throw $this->createNotFoundException('Audio introuvable.');
        }

        if (!str_starts_with($realFile, $realBase . DIRECTORY_SEPARATOR)) {
            throw $this->createNotFoundException('Audio introuvable.');
        }

        if (!is_file($realFile) || !is_readable($realFile)) {
            throw $this->createNotFoundException('Audio introuvable.');
        }

        $allowed = ['mp3', 'ogg', 'wav'];
        $ext = strtolower(pathinfo($realFile, PATHINFO_EXTENSION));
        if (!in_array($ext, $allowed, true)) {
            throw $this->createAccessDeniedException('Extension non
autorisée.');
        }

        return new BinaryFileResponse($realFile);
    }
}
```

Note: si on veut un player audio confortable (fichiers lourds, avance rapide, reprise a un endroit précis), il est préférable de gérer les requêtes HTTP de type Range. Elles permettent au navigateur de demander seulement une portion du fichier au lieu de tout télécharger. On peut toutefois commencer sans, tant que les fichiers restent raisonnables et qu'on accepte un confort moindre.

---

5 Empêcher l'accès direct Apache au dépôt (optionnel mais utile)

Si on veut forcer le passage par Symfony (même en Partie A), on bloque l'accès direct à `public/depot`.

Dans Apache, dans le vhost correspondant:

```
<Directory /var/www/TON_PROJET/public/depot>
    Require all denied
</Directory>
```

Résultat:

- `/depot/pages/hello.html` -> interdit
- `/p/hello` -> OK (car Symfony lit sur disque et renvoie)

---

6 Partie B: authentification et rôles

6.1 Principe

On distingue des routes publiques et des routes privées. Par exemple:

- tout le monde voit `/p/{slug}` et `/f/{filename}`
- seuls les connectés voient `/private/p/{slug}` et `/private/f/{filename}`
- seul `ROLE_ADMIN` voit `/admin/files/...`

6.2 Protéger dans le contrôleur (simple)

Dans une action, on peut refuser l'accès si le rôle n'est pas présent:

```
$this->denyAccessUnlessGranted('ROLE_USER');
```

Ou:

```
$this->denyAccessUnlessGranted('ROLE_ADMIN');
```

6.3 Exemple: pages privées stockées hors `public/`

On stocke dans `var/private/pages` (donc non accessible directement par URL).

```
#[Route('/private/p/{slug}', name: 'private_page', requirements: ['slug' =>
'[a-zA-Z0-9\-_]+' ])]
public function privatePage(string $slug): Response
{
    $this->denyAccessUnlessGranted('ROLE_USER');

    $projectDir = $this->getParameter('kernel.project_dir');
    $baseDir = $projectDir . '/var/private/pages';
```

```
        $file = $baseDir . '/' . $slug . '.html';

        $realBase = realpath($baseDir);
        $realFile = realpath($file);

        if ($realBase === false || $realFile === false) {
            throw $this->createNotFoundException('Page introuvable.');
        }

        if (!str_starts_with($realFile, $realBase . DIRECTORY_SEPARATOR)) {
            throw $this->createNotFoundException('Page introuvable.');
        }

        if (!is_file($realFile) || !is_readable($realFile)) {
            throw $this->createNotFoundException('Page introuvable.');
        }

        return new BinaryFileResponse($realFile);
    }
```

---

7 Bonnes pratiques Symfony pour une vitrine propre

7.1 Utiliser des routes lisibles

On adopte une convention simple:

- `/p/xxx` pages
- `/f/xxx.ext` fichiers
- `/audio/xxx.mp3` audio
- `/private/...` privé

7.2 Centraliser les chemins dans la config

Au lieu d'écrire les chemins dans chaque contrôleur, on peut définir des paramètres dans `config/services.yaml`:

```
parameters:
  depot_pages_dir: '%kernel.project_dir%/public/depot/pages'
  depot_files_dir: '%kernel.project_dir%/public/depot/files'
  depot_audio_dir: '%kernel.project_dir%/public/depot/audio'
  private_pages_dir: '%kernel.project_dir%/var/private/pages'
```

Puis on les récupère dans le code avec:

- `$this->getParameter('depot_pages_dir')`

7.3 Choisir entre HTML brut et Twig

Si on veut une page avec header/footer/menu commun, Twig est souvent plus adapté:

- `templates/pages/xxx.html.twig`
- un contrôleur qui fait `render()`

Les fichiers HTML bruts restent utiles pour:

- prototypes rapides
- mini-projets HTML/CSS/JS indépendants
- ressources externes déposées telles quelles

Exercices

Partie A (sans rôles)

Exercice A1 - Pages HTML statiques via Symfony

1. On crée le dossier: `public/depot/pages/`
2. On ajoute un fichier: `public/depot/pages/hello.html`
3. On crée le contrôleur `StaticPageController`
4. On vérifie:
    - `/p/hello` affiche la page

Bonus:

- On crée `public/depot/pages/cv.html` et on teste `/p/cv`

Exercice A2 - Ressources (PDF / images)

1. On crée `public/depot/files/`
2. On ajoute `reglement.pdf` et `logo.png`
3. On crée `StaticFileController`
4. On vérifie:
    - `/f/reglement.pdf`
    - `/f/logo.png`

Bonus:

- On modifie la whitelist pour autoriser `.webp` si besoin

Exercice A3 - Audio

1. On crée `public/depot/audio/`

2. On ajoute `demo.mp3`
3. On crée `AudioController`
4. On vérifie:
    - `/audio/demo.mp3`

Bonus HTML:

- On crée `public/depot/pages/player.html` avec:
    - `<audio controls src="/audio/demo.mp3"></audio>`
- On teste `/p/player`

Exercice A4 - Option sécurité Apache (bloquer accès direct)

1. On ajoute la règle Apache pour interdire `/depot`
2. On recharge Apache
3. On vérifie:
    - `/depot/pages/hello.html` -> interdit
    - `/p/hello` -> OK

Partie B (avec rôles)

Pré-requis B0 - Mettre en place le login

1. On installe et on configure Symfony Security (login form)
2. On crée un utilisateur de test:
    - user: `test@example.com`
    - password: `test`

On peut faire simple pour l'exercice: le but est l'autorisation, pas la BDD parfaite.

Exercice B1 - Route privée (ROLE_USER)

1. On crée `var/private/pages/secret.html`
2. On crée la route:
    - `/private/p/secret`
3. On ajoute dans l'action:
    - `denyAccessUnlessGranted('ROLE_USER')`

4. On vérifie:
    - non connecté -> 403 ou redirection login (selon la config)
    - connecté -> page OK

Exercice B2 - Fichier privé (ROLE_USER)

1. On crée `var/private/files/plan.pdf`
2. On crée la route:
    - `/private/f/plan.pdf`
3. On applique la même logique que les fichiers publics, avec en plus:
    - `denyAccessUnlessGranted('ROLE_USER')`
4. On vérifie en connecté et non connecté

Exercice B3 - Zone admin (ROLE_ADMIN)

1. On crée la route:
    - `/admin/files/{filename}`
2. On ajoute:
    - `denyAccessUnlessGranted('ROLE_ADMIN')`
3. On crée un user admin (ou on modifie le rôle)
4. On vérifie:
    - user normal -> refus
    - admin -> OK

Exercice B4 - Mix public + privé sur un même controller

But: `/p/{slug}` est public, mais si `slug` commence par `private-`, on impose une connexion.

1. On ajoute dans `page()`:
    - si `str_starts_with($slug, 'private-')` alors `denyAccessUnlessGranted('ROLE_USER')`
2. On dépose:
    - `public/depot/pages/private-notes.html`
    - `public/depot/pages/public-notes.html`
3. On vérifie:

    - `/p/public-notes` -> toujours OK
    - `/p/private-notes` -> seulement connecté

Annexe A - Apache2: DocumentRoot et réécriture vers Symfony

Dans un serveur web, on commence toujours par configurer une **racine**: le **DocumentRoot**.

- Le `DocumentRoot` est le dossier à partir duquel Apache cherche les fichiers à servir.
- Dans un projet Symfony, on pointe presque toujours vers: `.../monprojet/public/`

Exemple (idée générale):

```
DocumentRoot /var/www/monprojet/public
```

Conséquence directe:

- Si on demande `https://site/logo.png` et que le fichier existe dans `public/logo.png`, alors **Apache sert le fichier directement** (Symfony n'intervient pas).
- Si on demande `https://site/p/hello` et qu'il n'existe pas de fichier `public/p/hello`, Apache ne trouve rien à servir comme fichier statique. On utilise alors une réécriture (rewrite) pour envoyer la requête vers Symfony.

Le mécanisme classique est:

1. Apache essaie de servir un fichier statique sous `public/`.
2. Si le fichier n'existe pas, Apache redirige la requête vers `public/index.php`.
3. Symfony reçoit la requête et choisit le contrôleur à exécuter en fonction des routes.

Règle type (principe, pas à copier tel quel sans adapter):

```
RewriteEngine On

# Si le fichier ou le dossier existe, Apache le sert directement
RewriteCond %{REQUEST_FILENAME} -f [OR]
RewriteCond %{REQUEST_FILENAME} -d
RewriteRule ^ - [L]

# Sinon, on envoie vers le front controller Symfony
RewriteRule ^ index.php [L]
```

Point à retenir:

- `public/` = zone "servable" par Apache.
- hors `public/` = non accessible directement (sauf `Alias` explicite).
- Symfony n'intervient généralement que si Apache ne sert pas déjà un fichier statique.

Annexe B - Tester avec curl et comprendre avec les logs

On peut vérifier très vite si une ressource est servie par Apache directement ou si elle passe par Symfony.

1) Tester les réponses HTTP avec curl

Exemples:

```
# Ressource servie directement (si elle existe sous public/)
curl -I https://site/depot/pages/hello.html

# Ressource servie via Symfony (route / contrôleur)
curl -I https://site/p/hello
```

Ce qu'on observe:

- le code de réponse (200/301/403/404)
- les headers (parfois différents)
- le temps de réponse (souvent plus bas en statique direct)

2) Vérifier l'accès direct vs accès via contrôleur

Si on a bloqué l'accès direct à `public/depot` côté Apache (ex: `Require all denied`), alors:

```
# Doit échouer (403) si on a interdit l'accès direct à /depot
curl -I https://site/depot/pages/hello.html

# Doit fonctionner si Symfony sert la page via contrôleur
curl -I https://site/p/hello
```

3) Lire les logs Apache pour voir ce qui s'est passé

Sur Debian/Raspberry Pi OS, les logs sont souvent dans:

- `/var/log/apache2/access.log`
- `/var/log/apache2/error.log`

Exemples:

```
# Voir les dernières requêtes
sudo tail -n 50 /var/log/apache2/access.log

# Suivre en direct pendant les tests
sudo tail -f /var/log/apache2/access.log
```

Interprétation utile:

- Si l'accès est un fichier statique direct, Apache loggue une requête sur l'URL du fichier.
- Si l'accès passe par Symfony, Apache loggue aussi la requête, mais côté serveur Symfony c'est `index.php` qui est exécuté (visible indirectement via le comportement, et parfois via des logs applicatifs Symfony).

4) Lire les logs Symfony (si on active les logs)

En environnement dev, Symfony écrit généralement dans:

- `var/log/dev.log`

Exemple:

```
sudo tail -f var/log/dev.log
```

Point à retenir:

- Une requête servie en statique direct peut ne rien écrire dans les logs Symfony.
- Une requête qui passe par une route Symfony apparaîtra dans les logs applicatifs (selon le niveau de log et l'environnement).